

Laboratory Experiment – 4

Aim: To observe the execution behavior of a C program in memory using the GNU Debugger (GDB), and to examine program flow through breakpoints, step-wise execution, and runtime variable inspection.

Theory:

1. Introduction to GDB

The GNU Debugger, commonly referred to as GDB, is a powerful debugging utility that provides developers the ability to observe and control how programs run. It is widely used for tracking down errors and understanding memory behavior.

Key capabilities of GDB include:

- Placing breakpoints at specific lines or functions
- Executing the program one instruction at a time
- Inspecting memory locations and variable contents
- Monitoring function call sequences and return values

GDB requires programs to be compiled with debug symbols by using the `-g` flag during GCC compilation. This ensures that source code information is embedded in the executable.

2. Breakpoints and Program Control

A breakpoint is a marker set at a particular instruction to halt program execution, allowing inspection of state at that moment.

Frequently used GDB commands:

- `break <func_name>` → places a breakpoint at the given function
- `run (r)` → launches the program
- `next (n)` → advances to the following line without entering functions
- `step (s)` → steps into the called function
- `continue (c)` → resumes until the next breakpoint

3. Memory Layout and Variable Inspection

GDB reveals how variables occupy memory during execution. As an example, from our output:

- `arr=0x7ffffffddf0` indicates the starting memory address of the integer array
- Array elements are laid out contiguously in memory (sequential addresses)

By repeatedly issuing the `next (n)` command, the following changes can be tracked:

- The current loop counter and conditions
- Updates to accumulator and tracking variables (total, largest, etc.)
- Return values passed back from called functions

This stepwise inspection demonstrates how data evolves in memory throughout program execution.

4. Significance of GDB in Systems Programming

- Pinpoints logic errors and helps resolve bugs efficiently
- Provides visibility into actual memory usage and data layout
- Deepens understanding of program control flow
- Indispensable for low-level and system-level development work

Source Code:

```
#include <stdio.h>
#define N 10

void displayElements(int data[], int n) {
    printf("\nArray contents:\n");
    for(int i = 0; i < n; i++) {
        printf("%d ", data[i]);
    }
    printf("\n");
}

int computeTotal(int data[], int n) {
    int total = 0;
    for(int i = 0; i < n; i++) {
        total += data[i];
    }
    return total;
}

int locateMax(int data[], int n) {
    int largest = data[0];
    for(int i = 1; i < n; i++) {
        if(data[i] > largest) {
            largest = data[i];
        }
    }
    return largest;
}

int linearSearch(int data[], int n, int target) {
    for(int i = 0; i < n; i++) {
        if(data[i] == target) {
            return i;
        }
    }
    return -1;
}

void arrangeAscending(int data[], int n) {
    for(int i = 0; i < n - 1; i++) {
        for(int j = 0; j < n - i - 1; j++) {
            if(data[j] > data[j + 1]) {
                int hold = data[j];
                data[j] = data[j + 1];
                data[j + 1] = hold;
            }
        }
    }
}

int main() {
    int data[N], target;

    printf("Enter 10 integers:\n");
    for(int i = 0; i < N; i++) {
        scanf("%d", &data[i]);
    }
}
```

```

    displayElements(data, N);

    int total = computeTotal(data, N);
    printf("Total = %d\n", total);

    int largest = locateMax(data, N);
    printf("Largest = %d\n", largest);

    printf("Enter a value to search: ");
    scanf("%d", &target);

    int pos = linearSearch(data, N, target);
    if(pos != -1)
        printf("Value located at index %d\n", pos);
    else
        printf("Value not present\n");

    arrangeAscending(data, N);

    printf("\nSorted output:\n");
    displayElements(data, N);

    return 0;
}

```

GDB Session Output:

```

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./prog1...
(gdb) break computeTotal
Breakpoint 1 at 0x1248: file prog1.c, line 13.
(gdb) break locateMax
Breakpoint 2 at 0x1294: file prog1.c, line 22.
(gdb) break linearSearch
Breakpoint 3 at 0x12f5: file prog1.c, line 31.
(gdb) break arrangeAscending
Breakpoint 4 at 0x134a: file prog1.c, line 41.
(gdb) r
Starting program: /home/user/prog1
Enter 10 integers:
14
72
39
55
8
63
91
27
46
80

Array contents:
14 72 39 55 8 63 91 27 46 80

Breakpoint 1, computeTotal (data=0xffffffffddf0, n=10) at prog1.c:13
13 int computeTotal(int data[], int n) {
(gdb) n

```

```

14     int total = 0;
(gdb) n
15     for(int i = 0; i < n; i++) {
(gdb) n
16         total += data[i];
...
(gdb) n
main () at prog1.c:58
58     printf("Total = %d\n", total);
(gdb) n
Total = 495

Breakpoint 2, locateMax (data=0x7fffffffddf0, n=10) at prog1.c:22
22 int locateMax(int data[], int n) {
(gdb) n
...
(gdb) n
Max = 91

Breakpoint 3, linearSearch (data=0x7fffffffddf0, n=10, target=0) at prog1.c:31
...
(gdb) n
Value not present

Breakpoint 4, arrangeAscending (data=0x7fffffffdea0, n=21845) at prog1.c:41
41 void arrangeAscending(int data[], int n) {
(gdb) n
...
(gdb) n
main () at prog1.c:76
76     printf("\nSorted output:\n");
(gdb) n
Sorted output:
77     displayElements(data, N);
(gdb) n

Array contents:
8  14  27  39  46  55  63  72  80  91
79     return 0;
(gdb) n
80 }
(gdb) n
__libc_start_main (main=0x555555554128 <main>, argc=1, argv=0x7fffffffdf08,
init=<optimized out>, fini=<optimized out>, rtld_fini=<optimized out>,
stack_end=0x7fffffffdef8) at ../csu/libc-start.c:342

```

Conclusion:

Through this experiment, we successfully used GDB to trace the execution of a C program involving array operations. We observed how breakpoints pause execution at critical function entries, how variables are stored and updated in memory at each step, and how GDB session output reveals the actual runtime behavior of sorting, searching, and arithmetic operations on integer arrays.